Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 21 - PAO 2026 – II

Agenda

1. Revisión Lección #3
2. Métricas de evaluación de modelos en ML-DL
   2.1 Matriz de Confusión
   2.2 Precisión y Recuperación (Precision & Recall)
1. Otras técnicas para resolver problemas de IA
   3.1 Comportamiento inteligente
   3.2 Características de las soluciones inteligentes

Técnicas para resolver problemas en IA

- Estrategias de búsqueda a ciegas:
    - Generate and Test
    - Breadth First
    - Depth First
    - Uniform cost search
    - Limited depth
    - Interactive depth
    - Bidirectional search

Generate and Test

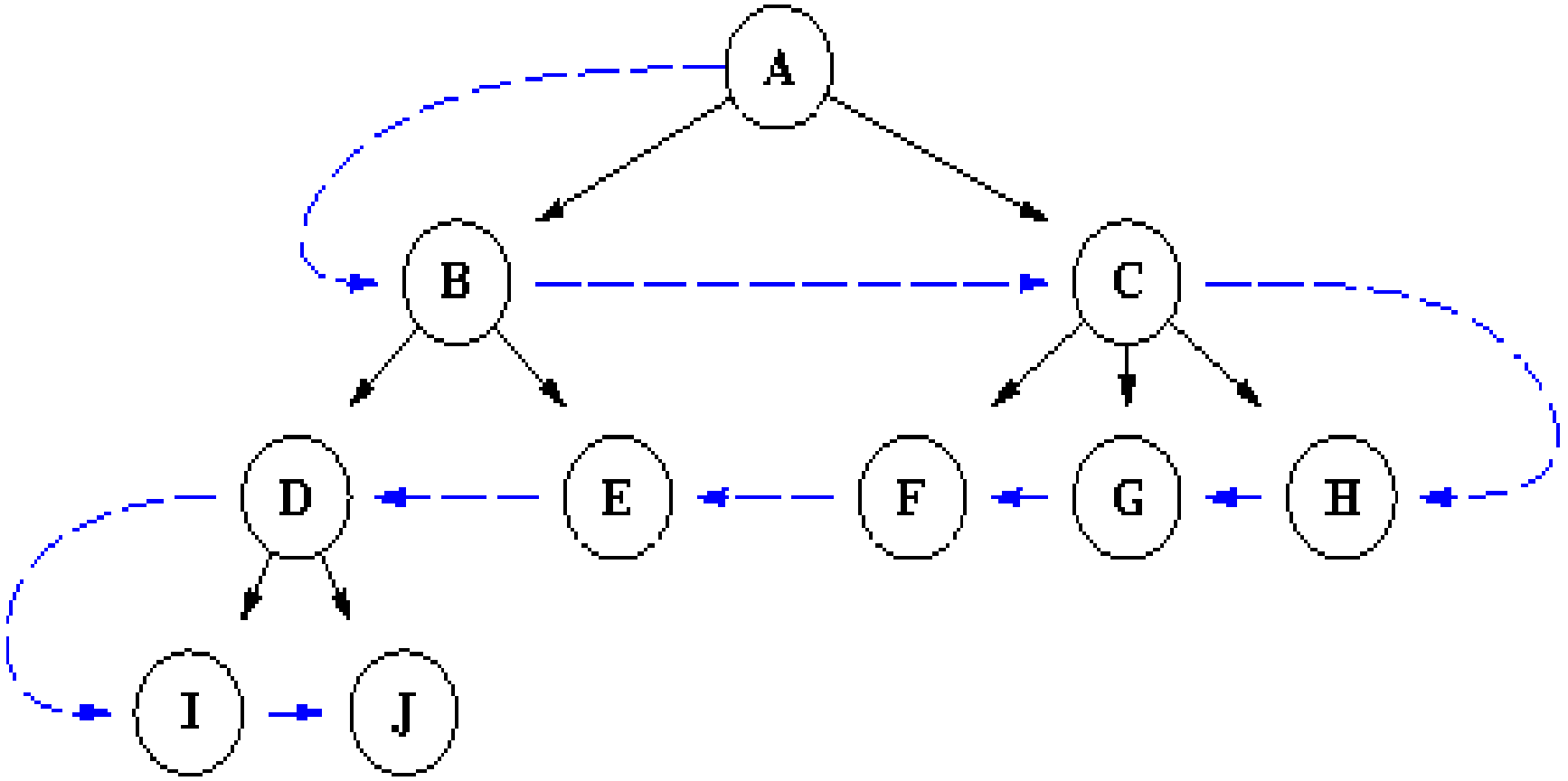
- Algoritmo

1. Genera una posible solución (estado o ruta)
2. Prueba para ver si es la solución. Compara con los elementos de un conjunto de soluciones aceptables.
3. Si se ha encontrado una solución, regrese. Si no, vaya al paso 1.

Generate and Test

- Una mejora del algoritmo es garantizar que una solución no se pruebe más de una vez. Esto se puede hacer enumerando las soluciones y probándolas en orden.
- Si las soluciones no se pueden enumerar, genere las soluciones al azar, pero mantenga una lista o una matriz con todas las soluciones probadas, para buscar una solución potencial antes de probarla.
- Esta alternativa tiene algunos problemas. Puede llevar tiempo buscar una solución probada, cuando la lista crece.
- Si el conjunto de posibles soluciones es grande, una mejor opción es generar aleatoriamente una solución sin probar.

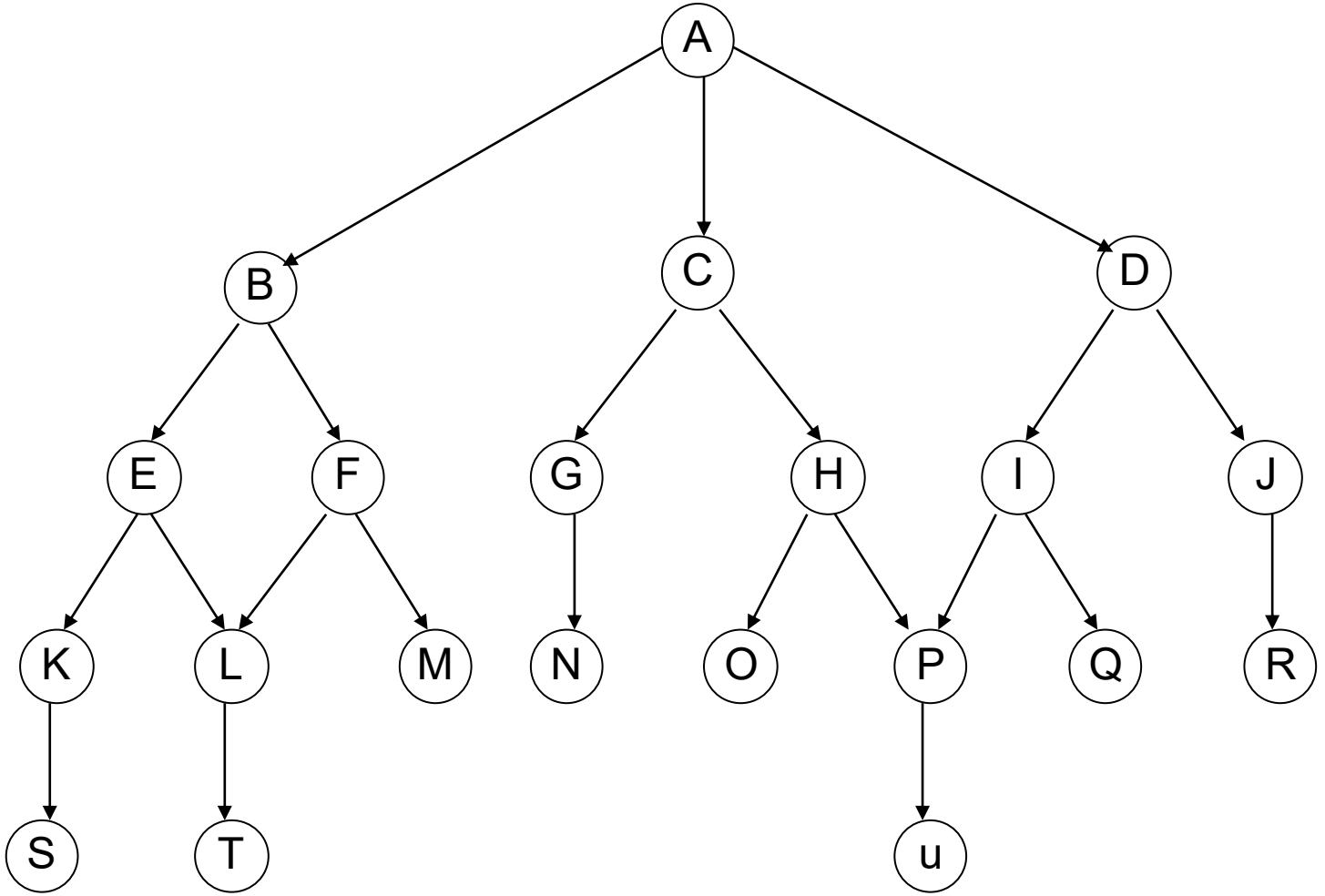
Breath - First



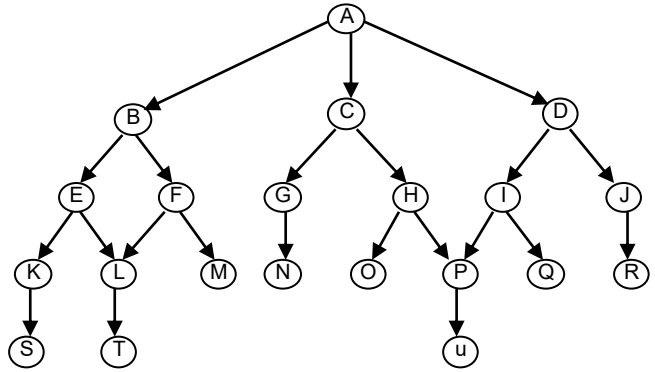
Breath - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on right end of open
                    end
        end
    return FAIL
end
```

Breath - First



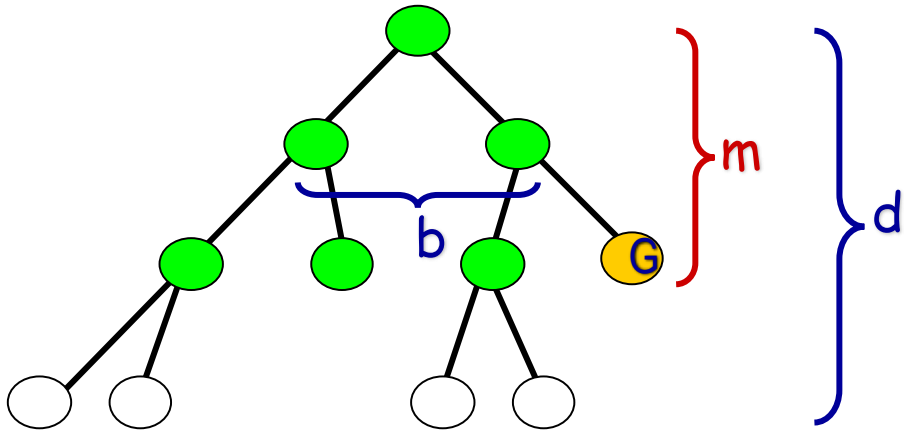
Breath - First



1. open = [A]; closed = [ ]
2. open = [B,C,D]; closed = [A]
3. open = [C,D,E,F]; closed = [B,A]
4. open = [D,E,F,G,H]; closed = [C,B,A]
5. open = [E,F,G,H,I,J]; closed = [D,C,B,A]
6. open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]
7. open = [G,H,I,J,K,L,M] L is in open; closed = [F,E,D,C,B,A]
8. open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]
9. Continue until U is found, or open = [ ].

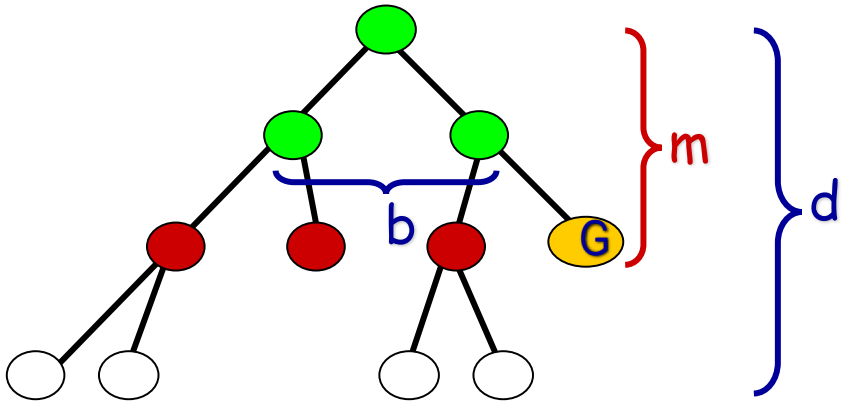
Breath - First

- Completo
  - ✓Si, (inclusive para arboles infinitos)
  - ✓Siempre encuentra la ruta más corta
- Tiempo
  - ✓$O(b^m)$

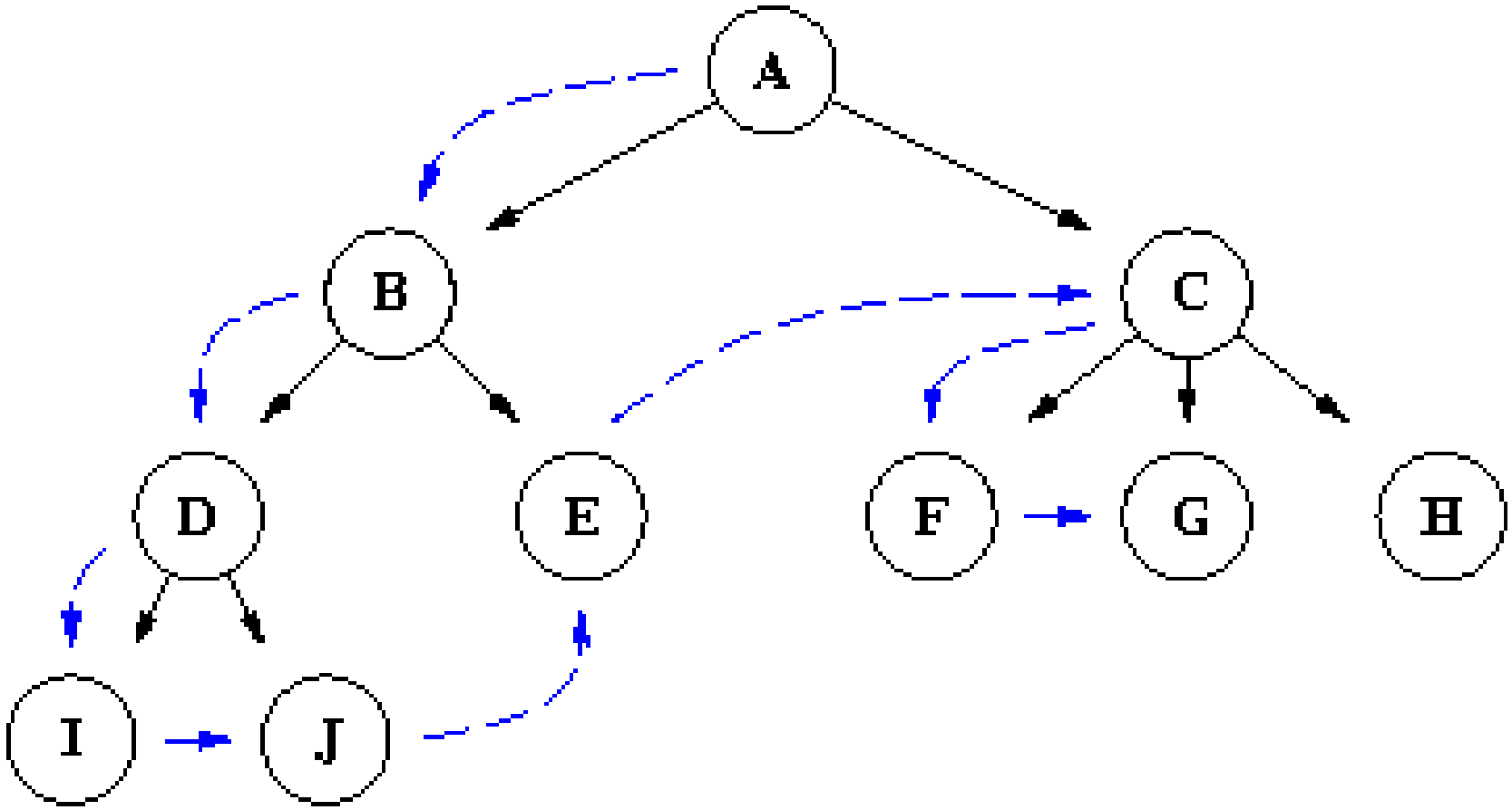


Breath - First

- Memoria
  - El número máximo de nodos se alcanza en el nivel "m" (objetivo)
  - En general: b^m
  - Generalmente "peor" que la profundidad primero



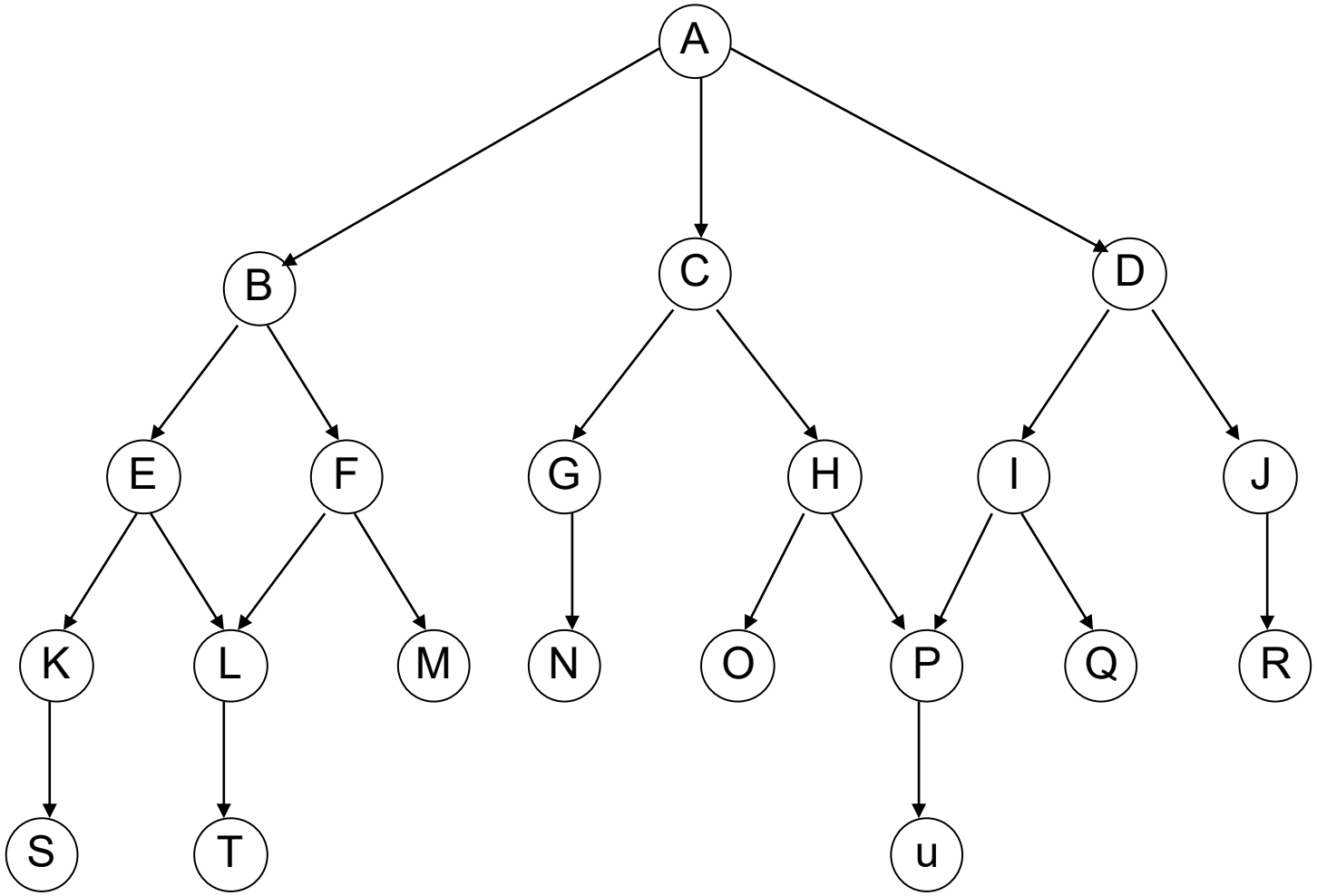
Depth - First



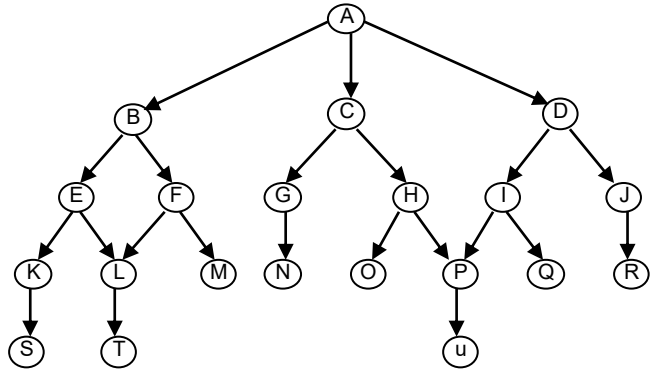
Depth - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on left end of open
                    end
        end
    return FAIL
end
```

Depth - First



Depth - First

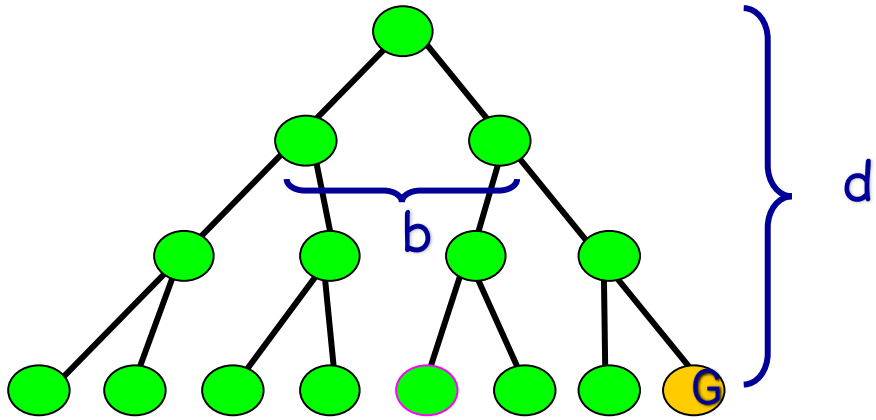


1. open = [A]; closed = [ ]
2. open = [B,C,D]; closed = [A]
3. open = [E,F,C,D]; closed = [B,A]
4. open = [K,L,F,C,D]; closed = [E,B,A]
5. open = [S,L,F,C,D]; closed = [K,E,B,A]
6. open = [L,F,C,D]; closed = [S,K,E,B,A]
7. open = [T,F,C,D]; closed = [L,S,K,E,B,A]
8. open = [F,C,D]; closed = [T,L,S,K,E,B,A]

Continua hasta encontrar U, o open = [ ].

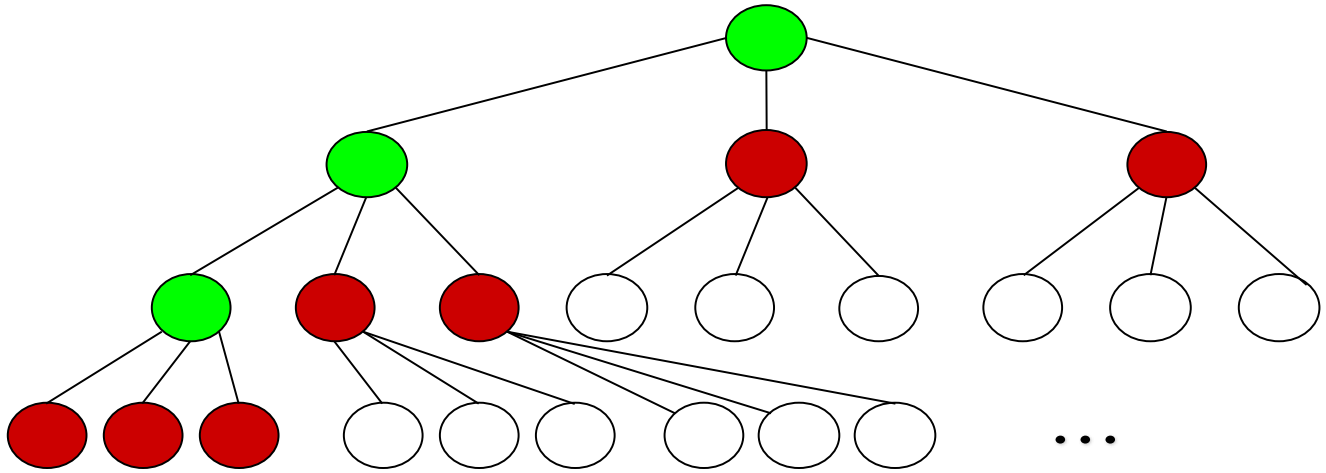
Depth - First

- Completo
  - Si -> árboles infinitos
- Tiempo
  - En el peor de los casos, el estado final podría ser la última rama del árbol.
  - $O(b^d)$



Depth - First

- Memoria
  - ✓En general: O(d*b)



Búsqueda heurística

- Entonces, utilizamos heurística para ”seleccionar" qué nodo expandir o explorar a continuación.
- La información heurística aparece como una función, que se evalúa con base en información específica para el contexto o problema en cuestión.
- Por lo tanto, la solución del problema de búsqueda consiste en maximizar o minimizar una función.
- La función heurística nos permite evaluar un nodo "localmente", en función del costo estimado para alcanzar el objetivo desde el nodo actual.
- Pero, hay algunos retos en la búsqueda con heurística:
  - La heurística podría no ayudar a encontrar la mejor solución: problema abierto. Sin embargo, el algoritmo A * lo resuelve con una función heurística h*
  - …

Hill Climbing

- ***Hill Climbing***
  - Es una versión mejorada del algoritmo de generación y prueba, pero con heurística.
  - Desde el procedimiento de prueba, hay una retroalimentación para ayudar al generador de soluciones decidir, en qué dirección ir a continuación o si es necesario cambiar de dirección de búsqueda en el espacio de soluciones.
  - En este proceso, la búsqueda finaliza si no hay un próximo estado o un nodo razonable para moverse.

Simple Hill Climbing

1. Evaluar el estado inicial.
2. Repetir, hasta que se encuentre una solución, o ya no haya nuevos operadores que aplicar:

   - Seleccionar un nodo candidato y aplicar la función heurística

   - Evaluar el nuevo estado:

     - nuevo estado = objetivo ? $\rightarrow$ terminar
     - nuevo estado es mejor que el estado actual ? $\rightarrow$ avanzar con el nuevo estado actual